

ANDROID STUDIO

KOTLIN-ALDAGAIK ETA FUNTZIOAK

HELBURUAK

Gai honetan kontzeptu hauek landuko ditugu:

1. Aldagaiak

- 1.1. Aldagaiak berresleitzea
- 1.2. Berresleitu ezin diren aldagaiak
- 1.3. Expresioak eta adierazpenak
- 1.4. Motak: Balio mota desberdinak
- 1.5. Aldagai mota orokorrak

2. Funtzioak

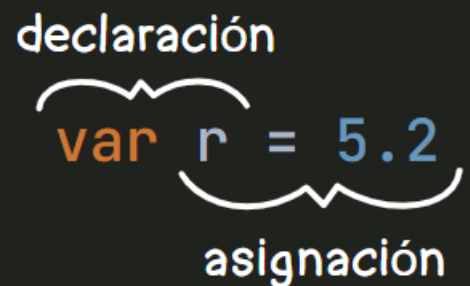
- 2.1. Funtzioen oinarrizko kontzeptuak

ALDAGAIK

Programazioan, aldagaiek proposita bakarra dute, balioak izatea. Kotlinen, aldagai bat sortu eta zenbaki bat jar dezakegu bertan, adibidez:

```
var r = 5.2
```

Kode hau bi gauza egiten ari da lerro batean.



```
declaración  
var r = 5.2  
asignación
```

ALDAGAIAK

- **Adierazpena:** `Var r` idazten dugunean, `r` izeneko aldagai bat deklaratzeko ari gara. Aldagai bat deklaratzeko balde bat sortzea bezalakoa dela pentsa dezakezu.
- **Esleipena:** `r = 5.2` idazten dugunean, `r` aldagaiari `5.2` balioa esleitzen ari gara. Bestela esanda, `5.2` zenbakia baldean jarriko dugu.



ALDAGAIAK

A diagram illustrating the components of a Kotlin variable declaration. The code snippet `var r = 5.2` is shown with annotations in Spanish. A dashed arrow points from the text 'nombre de la variable' to the variable name 'r'. Another dashed arrow points from 'palabra reservada' to the keyword 'var'. A third dashed arrow points from 'valor' to the value '5.2'.

```
nombre de la variable  
→ var r = 5.2 ←  
palabra reservada  valor
```

Kode-lerro honen hiru zatiak zatituko ditugu.

- ▶ **Var:** Kotlini aldagai bat sortzeko esaten dion hitz erreserbatua da
- ▶ **r:** aldagaiaren izena edo identifikatzailea da.
- ▶ **5.2 zenbakia:** aldagaiari esleitzen zaion balioa da.

ALDAGAIAK

var da Kotlin ikasten jarraitzen dugunean ikusiko dugun gako-hitz ugarietako bat. Jakin behar den gauzarik garrantzitsuena da ezin duzula **var** erabili **aldagai bat** izendatzeko. Adibidez, zuk ezin duzu var erabili aldagai baten izen gisa:

```
var_var = 5
```

Listado 1.2 - **Error:** "Expecting property name or receiver type" (No compila)

ALDAGAIAK BERRRESLEITZEA

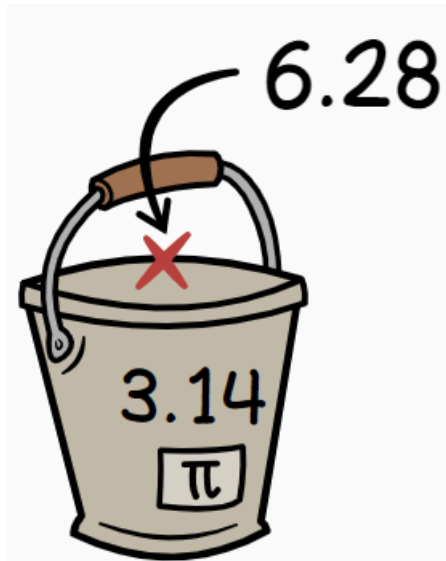
Aldagai bat **var** bezala definitzen badugu, aldagaiari nahi beste alditan berresleitu ahal izango diogu balio.

```
var r = 5.2  
r = 6.7
```

Aldagai hori berresleitzen dugunean, guk ez dugu var gako-hitza berriro erabili beharrik, aldagaia lehen leerroan deklaratu baitzen.

BERRESLEITU EZIN DIREN ALDAGAIK

Kotlinen programatzen dugunean, une batzuetan ez dugu nahi aldagai bat aldatzea.



BERRESLEITU EZIN DIREN ALDAGAIAK

Kasu horietan, **var**-ekin deklaratu ordeztu **val**-ekin deklaratzeko duzu, honela:

```
val π = 3.14
```

Orain, berriz esleitu nahi baduzu, errore bat izango duzu:

```
val π = 3.14  
π = 1.23
```

Listado 1.5 - **Error:** "Val cannot be reassigned" - Val no puede ser re-asignado. (No compila)

BERRESLEITU EZIN DIREN ALDAGAIAK

Zure aldagaiak val-ekin adieraztea modu egokia da aldatu behar ez litzatekeen zerbaiten balioa aldatzen ez duzula bermatzeko. Izan ere, ideia ona da beti val-arekin hastea, eta behar denean aldatzea.

ALDAGAIK IZENDATU

Batzuetan, hitz bat baino gehiago erabili behar izaten dugu aldagaiak identifikatzako. Kotlinen, ohitura da lehen hitza letra xehez hastea eta gainerako hitzak maiuskulaz egitea, hemen erakusten den bezala:

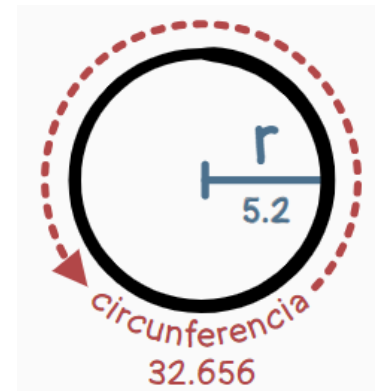
```
var radioDelCirculo = 5.2
```

Idea ona da zure kodea zure taldeko programatzaileen antzera idaztea; norbaitek kodea begiratzen duenean arazorik ez izateko ulertzeko.

EXPRESIOAK ETA ADIERAZPENAK

```
var radio = 5.2
val pi = 3.14

val circunferencia = 2 * pi * radio
```



Orain arte, 5.2 eta 3.14 balioak baino ez ditugu esleitu. Hau da zerbait konplexuagoa esleitzen dugun lehen aldia: $2 * \pi * \text{radio}$.

Kotlinek hori ikusten duenean, emaitza kalkulatzen du guretzat. Erradioa bider 2 aldiz pi biderkatzen du eta, ondoren, emaitza hartzen du eta zirkunferentzia izeneko aldagaiari esleitzen dio. Kasu honetan (5.2 erradioarekin), zirkunferentzia 32.656koa da.

MOTAK: BALIO MOTA DESBERDINAK

```
var radio = 5.2  
val pi = 3.14
```

Aldagai hauek kontuan izanda, erradioa eta pi-k puntu hamartar bat duten zenbakiak irudikatzen dituzte, eta Kotlin-ek Double deitzen dion tipo bat da. Nahi baduzu, Kotlini aldagaiaren mota zuk zeuk ere esan diezaiokezu, honela:

```
var radio: Double = 5.2  
val pi: Double = 3.14
```

MOTAK: BALIO MOTA DESBERDINAK

Hori egiten dugunean, aldagaiaren mota esplizituki zehazten ari gara. Askotan ez dugu hau egin behar, Kotlinek ahalik eta ondoen egiten du, esleitu diogun horretan oinarritutako tipoa inferitzeko. Prozesu horri **moten inferentzia** esaten zaio.

Orduan hau idazten dugunean...

```
var radio = 5.2
```

... orduan Kotlini esaten diogu 5.2 Double bat dela, eta automatikoki Double mota erabiltzen du erradioan.

Normalean eroso da Kotlinek mota-inferentzia gugatik egitea, baina batzuetan atsegina ere izango litzateke inferitzen den tipoa ikustea. Kotlinek ondorioztatzen duena ikusteko, IntelliJ eta Android Studiok Dokumentazio Azkarra izeneko ezaugarri bat du. Testu-kurtsorea aldagaiaren izenean jartzen dugu, eta teklatuaren metodo laburtua sakatzen du. Ctrl+Q Windows-en eta Linux-en.

```
var radius = 5.2
val pi = 3.14
val pi: Double
my-project.main
```

INTEGERRAK (OSOAK-INT)

Orain arte puntu hamartarra duten zenbakiak ikusi ditugu, hala nola 5.2 eta 3.14. Baina puntu hamartarra duen zenbaki bat ere erabil dezakegu, 5 eta 3 esaterako. Zenbaki mota horiei **Integerrak** deitzen zaie, eta Kotlinen **Int** bezala ezagutzen dira modu laburtuan. Hona hemen Integer aldagai bat sortzeko moduaren adibide bat:

```
val numeroDePerros: Int = 3
```


BOOLEARRAK(BOOLEAN)

Batzuetan, zuk nahi ditugun aldagaiak piztua edo itzalia, edo egia edo gezurra, etab. Izango dira kasu horietan, **Boolean** motako aldagai bat behar dugu.

```
val esLaTierraRedonda: Boolean = true  
val esLaTierraPlana: Boolean = false
```

STRING-AK (KATEAK)

- Guk ere gorde dezakegu testua aldagai batean. Programatzaile batentzat hori katea da, karaktere pila bat delako, hala nola letrak, zenbakiak eta sinbolo "elkartuak". Kotlinen, kateetarako motari `String` deitzen zaio, eta guk `String` aldagai bat sor dezakezu, honela:

```
val texto: String = "Esto es una cadena"
```

STRING-AK (KATEAK)

Guk ere gorde dezakegu testua aldagai batean. Programatzaile batentzat hori katea da, karaktere pila bat delako, hala nola letrak, zenbakiak eta sinbolo "elkartuak". Kotlinen, kateetarako motari `String` deitzen zaio, eta guk `String` aldagai bat sor dezakezu, honela:

```
val texto: String = "Esto es una cadena"
```

Hauek dira aldagai mota ohikoenak. Irakasgai honekin jarraitu ahala, ikusiko dugu nola klaseak esaterako, ikusi berri ditugun oinarrizko mota horietan oinarrituta.

FUNTZIOAK

Erreza da zirkulu baten zirkunferentzia kalkulatzea ondorengo formulekin:

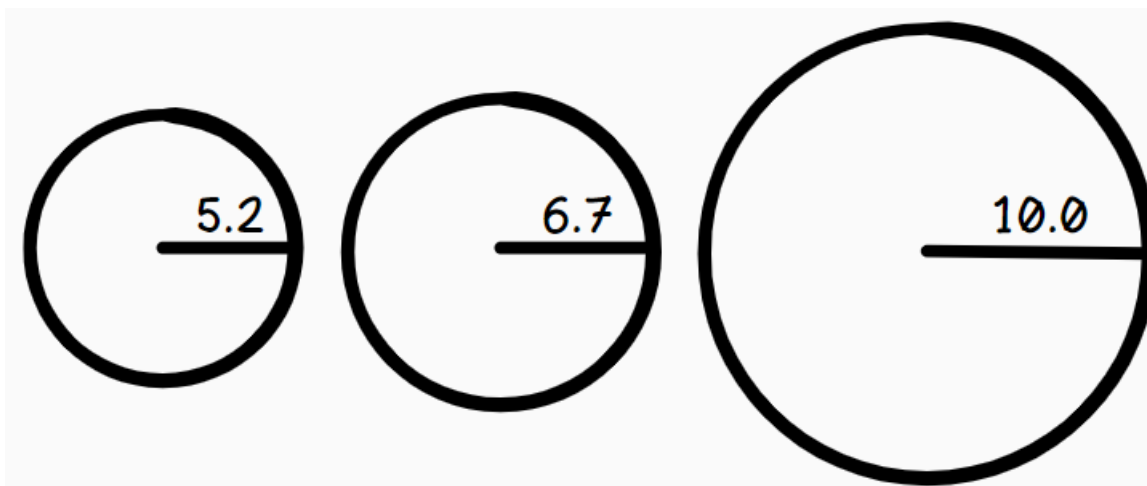
$$\text{Zirkunferentzia} = 2 \times \pi \times r$$

Hona hemen kalkulu hori egiteko idatzi beharreko Kotlin kodea:

```
val pi = 3.14  
var radio = 5.2  
val circunferencia = 2 * pi * radio
```

FUNTZIOAK

Kode honek 5,2 erradioko zirkulu baten zirkunferentzia kalkulatu du. Jakina, zirkulu guztiek ez dute 5,2ko erradioa! Zer gertatzen da 6,7 edo 10,0ko erradioa duen zirkulu baten zirkunferentzia zehaztu nahi badugu?



FUNTZIOAK

Bueno, ekuazio asko idatz genitzake:

```
val pi = 3.14
var radio = 5.2
val circunferenciaDeCírculoPequeño = 2 * pi * radio

radio = 6.7
val circunferenciaDeCírculoMediano = 2 * pi * radio
radio = 10
val circunferenciaDeCírculoGrande = 2 * pi * radio
```

FUNTZIOAK

Honek funtzionatzen du duda
barik, baina programatzeko era
egokiena da?

FUNTZIOAK

Kode zati bera toki ezberdinetan dugunean, bikoizketa dugula esaten dugu. Kasu gehienetan kode bikoiztua txarra da, arrazoi hauengatik:

- ▶ Gauza bera askotan tipifikatzen dugunean, lehenago edo geroago gaizki idatziko dugu kasu batean.
- ▶ Ekuazioa aldatu nahi badugu, idatzi genuen leku guztiak bilatu beharko ditugu, eta horietako bakoitza eguneratzen dugula ziurtatu.
- ▶ Zailagoa izan daiteke irakurtzeko gure begiek pantailan behin eta berriz kode bera idatzita ikusten dutenean.

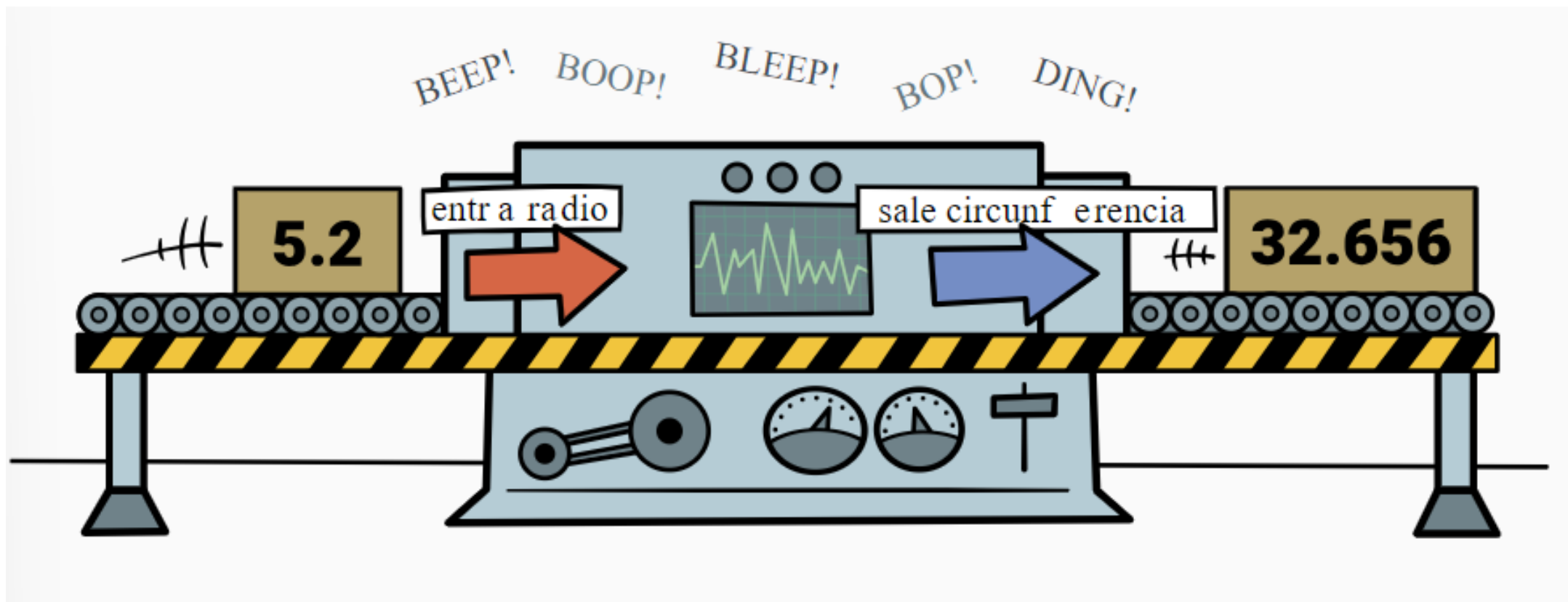
FUNTZIOAK

Gure kodea aldatuko dugu, $2 \cdot \pi \cdot \text{erradioa}$ behin bakarrik ikus dezagun, eta orduan hori erabili edozein zirkuluren zirkunferentzia kalkulatu. Bestela esanda, bikoizketa ezabatuko dogu!

$2 \cdot \pi \cdot \text{erradioa}$ hiru aldiz idatzi dugun arren, aldiro aldatzen dugun balio bakarra erradioa da. Beste era batera esanda, 2 ez da inoiz aldatu, eta π ez da inoiz aldatu (aldi bakoitzean 3.14 da). Baina erradioaren balioaaldi bakoitzean ezberdina izan da: lehenik 5.2, gero 6.7 eta gero 10.0.

Erradioa aldatzen den gauza bakarra denez, primerakoa litzateke erradioa zirkunferentzia bihurtzea. Bestela esanda, nolakoa izango litzateke makina batean erradioa sartuko balitz eta zirkunferentzia itzuli?

FUNTZIOAK



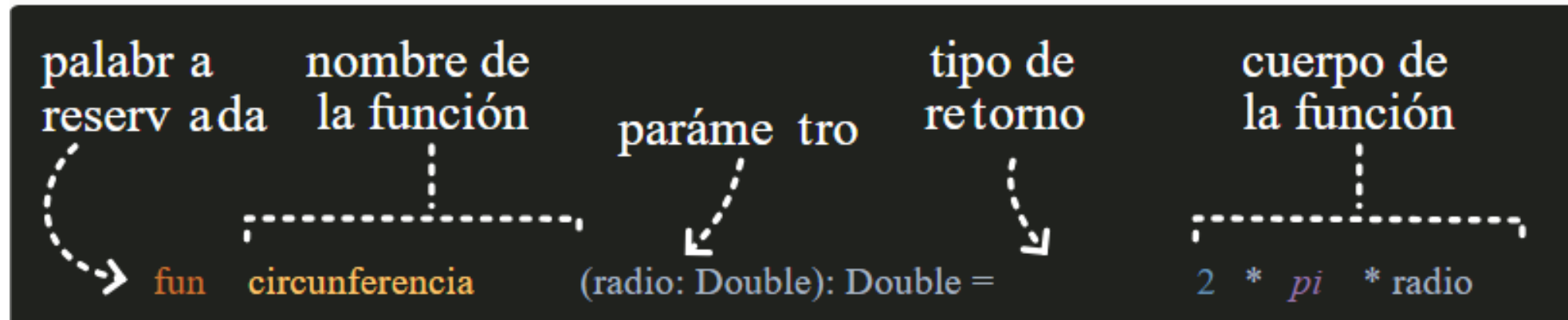
FUNTZIOAK

Erradioa alde batean jartzen dugunez, honi **sarrera** dei geniezaioke eta zirkunferentzia beste aldetik ateratzen denez, horri **irteera** dei geniezaioke. Orain, ez dugu makina bat sortu behar, baina **funtzio** bat sor dezakegu haren ordeez. Funtzio horrek nahi duguna egingo du, erradio bat emango diogu eta zirkunferentzia bat itzuliko digu.

FUNTZIOEN OINARRIZKO KONTZEPTUAK

Funtzio bat sortzen

Hemen, Kotlinen funtzio nola adierazioko litzatekeen erakusten dugu:



FUNTZIOEN OINARRIZKO KONTZEPTUAK

Informazio asko dirudi, baina zati gutxi daude eta ulerterrazak dira.

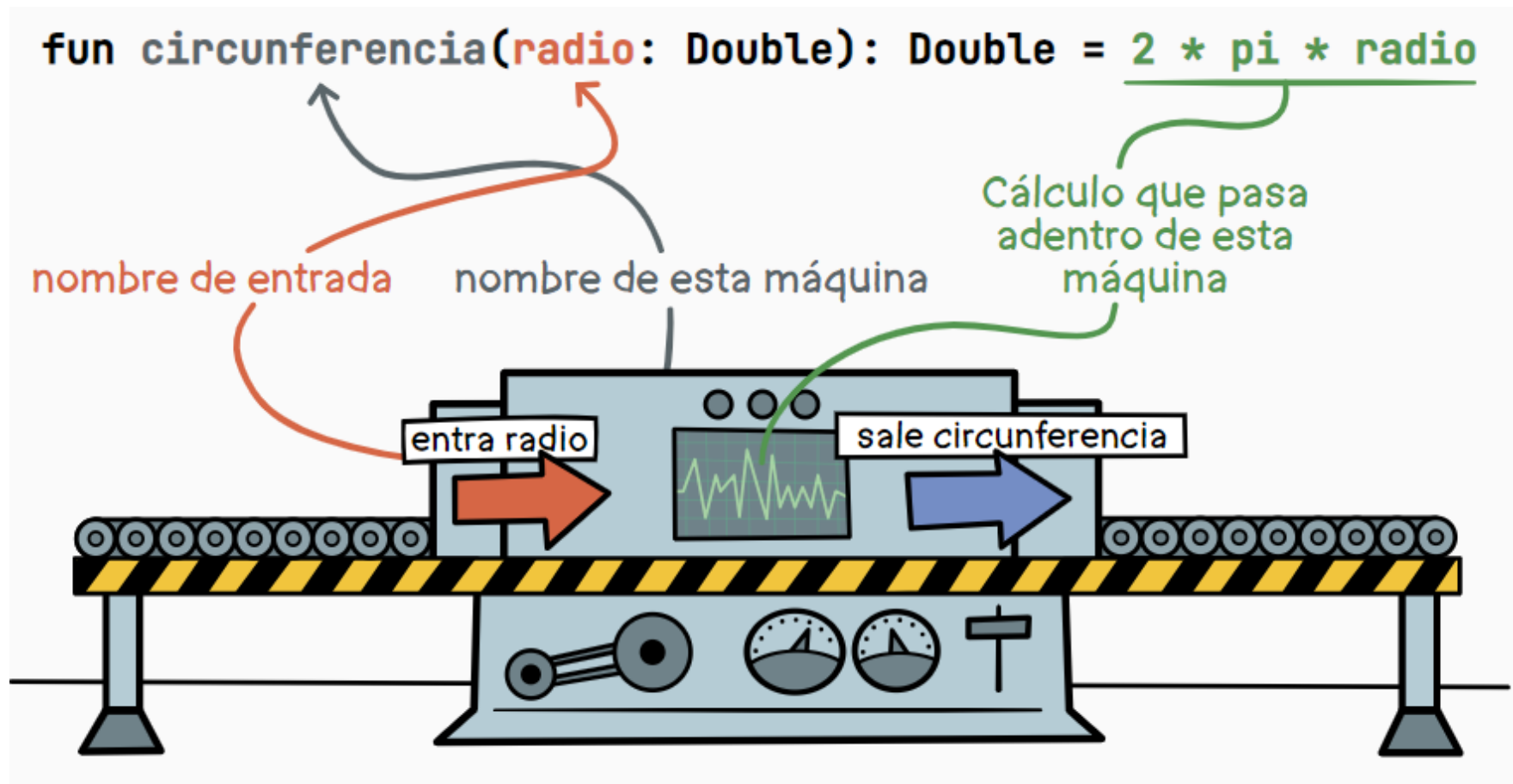
1. Lehenik eta behin, **fun** hitz erreserbatua da (**val** eta **var** hitz erreserbatuak diren moduan). Horrek esaten dio Kotlini funtzio bat idazten ari zarela.
2. Jarraian, **zirkunferentzia** funtzioaren izena da. Gure funtzioa edozein izenekin izenda genezake, baina zirkunferentzia aukeratu dugu.
3. Ondoren, (**radio: Double**) funtzioaren prametroak adieraziko dira, sarrerako balioa kasu hontan, **radio** izenekoa, eta **Double** motakoa izango da.

FUNTZIOEN OINARRIZKO KONTZEPTUAK

4. Ondorengo, :Double parentesien ondoren funtzioaren irteera mota adierazten du, kasu honetan Double. Funtzioaren itzulera-mota deitzen zaio horri.
5. Eta azkenik, $2 * \pi * \text{radio}$, funtzioaren gorputza deitzen da. Eragiketa horren emaitza funtzioaren irteera-balioa izango da. Irteera-balioa funtzioaren emaitza da. Gure makinatik ateratzen dena da. Kontuan izan behar dugu itzulera-mota eragiketaren itzulera-mota bera izan behar duela.

Hurrengo irudian, gure funtzioa eta goian irudikatzen dugun makina alderatzen ditugu.

FUNTZIOEN OINARRIZKO KONTZEPTUAK



FUNTZIOEN OINARRIZKO KONTZEPTUAK

Lehen ikusi dugun bezala, ez dugu beti aldagaiaren mota zehaztu behar; horren ordez, Kotlinek bere tipo-inferentzia erabiltzen du. Guk ere tipoen inferentzia erabil dezakezu honelako funtzioak idazten ditugunean. Itzulera-mota aldatu besterik ez du egiten, honela ikus dadin:

```
fun circunferencia(radius: Double) = 2 * pi * radius
```


FUNTZIOEN OINARRIZKO KONTZEPTUAK

Funtzio bati deitu

Funtzioa erabiltzen duzunean, hau da, makinan zerbait jartzen duzunean, **funtzioari deitu** esaten zaio(invocar). Deitzen den lekuari **dei-kodea** edo **dei-lekua** deitzen zaio.

Kotlin-en funtzio bati honela deitzen zaio:

```
val circunferenciaDelCírculoPequeño = circunferencia(5.2)
```

The diagram illustrates the components of the function call `circunferencia(5.2)` within the assignment statement. Three labels with dashed arrows point to specific parts of the code: 'variable' points to `val`, 'nombre de la función' points to `circunferencia`, and 'argumento' points to `5.2`.

FUNTZIOEN OINARRIZKO KONTZEPTUAK

1. Lehenik eta behin, **circunferenciaDelCírculoPequeño** funtzioaren emaitza jasotzen duen aldagai bat da (hau da, makinatik ateratzen dena)
2. Ondoren, **zirkunferentzia** deitzen ari garen funtzioaren izena da.
3. **5.2** balioa funtzioaren argumetua da, makinaren barruan jartzen dugun gauza da. Funtzioari argumentu batekin deitzen diogunean, batzuetan argumentua funtziora pasatzen ari garela esaten dugu.

FUNTZIOEN OINARRIZKO KONTZEPTUAK

Zer gertatzen da funtzioari deitzen diogunean? Esan dezagun funtzio bat idatzi eta honela deitzen diogula:

```
fun circumferencia(radio: Double) = 2 * pi * radio  
  
val circumferenciaDelCírculoPequeño = circumferencia(5.2)
```

FUNTZIOEN OINARRIZKO KONTZEPTUAK

Funtzioari deitzen diogunean, $2 * \pi * \text{erradioko}$ funtzioaren gorputza jartzea bezala da, non funtzioari `circunferencia(5.2)` deitzen zaio.

Orduan honela dela pentsa genezake:

```
fun circunferencia(radio: Double) = 2 * pi * radio  
val circunferenciaDelCírculoPequeño = 2 * pi * radio
```

Gero, erradio gisa 5.2 pasatu dugunez, pentsa genezake erradioa jarri genuen lekuan 5.2 balioa ordeztuko genuela:

```
fun circunferencia(radio: Double) = 2 * pi * radio  
val circunferenciaDelCírculoPequeño = 2 * pi * 5.2
```

FUNTZIOEN OINARRIZKO KONTZEPTUAK

Laburbilduz, **circunferencia** (5.2) deitzen dugunean, leku berean $2 * \pi * 5.2$ idatziko bagenu bezala da.

Funtzioak erradioa erabiliz zirkunferentzia kalkula dezakeenez, gure funtzioari behar dugun aldi guztietan dei diezaiokegu!

```
val pi = 3.14
fun circunferencia(radius: Double) = 2 * pi * radius

val circunferenciaDeCírculoPequeño = circunferencia(5.2)
val circunferenciaDeCírculoMediano = circunferencia(6.7)
val circunferenciaDeCírculoGrande = circunferencia(10.0)
```

FUNTZIOEN OINARRIZKO KONTZEPTUAK

Argumentuak eta Parametroak: Zein da aldea?

Erraza da argumentu bat parametro batekin nahastea; horregatik, garrantzitsua da desberdintasuna ulertzea:

- ▶ **Argumentu** bat funtziora pasatzen diogun balio bat da. Adibidez, 5.2 pasatu nahi baduzu `circunferencia()` funtziora. Argumentua 5.2 da.
- ▶ **Parametro** bat, funtzioaren barruko aldagaia da, balioa bat izango duena. Adibidez, 5.2 `circunferencia()` igarotzen garenean, erradioa izeneko parametroari esleitzen zaio.

Argumentu bat funtziotik **kanpo** etorri ohi den **balio** bat da.

FUNTZIOEN OINARRIZKO KONTZEPTUAK

Parametro bat baino gehiago dituzten funtzioak

circunferencia() funtzioak radio izeneko parametro bakarra du, baina batzuetan funtzio batek parametro bat baino gehiago behar du. Bi parametro dituen funtzio bat sortuko dugu.

$$\text{velocidad} = \frac{\text{distancia}}{\text{tiempo}}$$

FUNTZIOEN OINARRIZKO KONTZEPTUAK

```
val distancia = 321.8  
val tiempo = 4.15  
val velocidad = distancia / tiempo
```

```
fun velocidad(distancia: Double, tiempo: Double) = distancia / tiempo
```

```
val velocidadPromedio = velocidad(321.8, 4.15)
```


FUNTZIOEN OINARRIZKO KONTZEPTUAK

```
fun velocidad(distancia: Double, tiempo: Double) = distancia / tiempo
```

```
fun velocidadPromedio = velocidad(321.8, 4.15)
```



The diagram consists of two dashed white arrows pointing from the arguments in the function call to the parameters in the function definition. One arrow starts at the value '321.8' in the call and points to the parameter 'distancia' in the definition. The other arrow starts at the value '4.15' in the call and points to the parameter 'tiempo' in the definition.

FUNTZIOEN OINARRIZKO KONTZEPTUAK

Izendatutako argumentuak

Argumentuen posizioan konfiantza izan beharrean, parametroen izena erabil dezakezu, honela:

```
val velocidadPromedio = velocidad(distancia = 321.8, tiempo = 4.15)
```

Honeri **izendatutako argumentuak** deitzen zaie. Aipatutako Argumentuen gauzarik bikainena da ordenak ez duela axola. Horregatik, funtzioari dei diezaiokezu argumentuak ordena desberdinean erabiliz:

```
val velocidadPromedio = velocidad(tiempo = 4.15, distancia = 321.8)
```

FUNTZIOEN OINARRIZKO KONTZEPTUAK

Izendatutako argumentuak

Argumentuen posizioan konfiantza izan beharrean, parametroen izena erabil dezakezu, honela:

```
val velocidadPromedio = velocidad(distancia = 321.8, tiempo = 4.15)
```

Honeri **izendatutako argumentuak** deitzen zaie. Aipatutako Argumentuen gauzarik bikainena da ordenak ez duela axola. Horregatik, funtzioari dei diezaiokezu argumentuak ordena desberdinean erabiliz:

```
val velocidadPromedio = velocidad(tiempo = 4.15, distancia = 321.8)
```

FUNTZIOEN OINARRIZKO KONTZEPTUAK

Beste era batera esanda, ondorengo funtzioen bost dei guztiak emaitza bera izango dute:

```
val velocidadPromedio1 = velocidad (321.8, 4.15)
val velocidadPromedio2 = velocidad (distancia = 321.8, 4.15)
val velocidadPromedio3 = velocidad (321.8, tiempo = 4.15)
val velocidadPromedio4 = velocidad (distancia = 321.8, tiempo = 4.15)
val velocidadPromedio5 = velocidad (tiempo = 4.15, distancia = 321.8)
```

FUNTZIOEN OINARRIZKO KONTZEPTUAK

Aurrez zehaztutako argumentuak

Demagun ondorengo datuak ditugula:

```
val velocidadCaminando = velocidad(10.2, 2.0)
val velocidadEnBicicleta = velocidad(29.6, 2.0)
val velocidadManejando = velocidad(225.3, 2.0)
val velocidadVolando = velocidad(1368.747, 1.5)
```

FUNTZIOEN OINARRIZKO KONTZEPTUAK

Deiak ondo aztertuz gero, denboraren parametroa 2.0 balioa duela behin eta berriz konturatuko gara, beraz, 2.0-a erabil dezakegu funtzioa definitzean aurrez zehaztutako argumentu gisa. Velocidad() funtzioa eguneratuz denbora-parametrorako 2.0 balio lehenetsia duena ondorengoa da:

```
fun velocidad(distancia: Double, tiempo: Double = 2.0) = distancia / tiempo
```

Orain denboraren argumentua alde batera utz dezakegu balioa 2.0ren berdina denean:

```
val velocidadCaminando = velocidad(10.2)
val velocidadEnBicicleta = velocidad(29.6)
val velocidadManejando = velocidad(225.3)
val velocidadVolando = velocidad(1368.747, 1.5)
```

FUNTZIOEN OINARRIZKO KONTZEPTUAK

Deiak ondo aztertuz gero, denboraren parametroa 2.0 balioa duela behin eta berriz konturatuko gara, beraz, 2.0-a erabil dezakegu funtzioa definitzean aurrez zehaztutako argumentu gisa. Velocidad() funtzioa eguneratuz denbora-parametrorako 2.0 balio lehenetsia duena ondorengoa da:

```
fun velocidad(distancia: Double, tiempo: Double = 2.0) = distancia / tiempo
```

Orain denboraren argumentua alde batera utz dezakegu balioa 2.0ren berdina denean:

```
val velocidadCaminando = velocidad(10.2)
val velocidadEnBicicleta = velocidad(29.6)
val velocidadManejando = velocidad(225.3)
val velocidadVolando = velocidad(1368.747, 1.5)
```

FUNTZIOEN OINARRIZKO KONTZEPTUAK

Aldez aurretik zehaztutako argumentu lehenengo argumentua daba

Demagun ondorengo datuak ditugula:

```
val velocidadCaminando = velocidad(42.195, 8.27)
val velocidadEnBicicleta = velocidad(42.195, 2.85)
val velocidadManejando = velocidad(42.195, 0.37)
val velocidadVolando = velocidad(0.12, 0.01)
```


FUNTZIOEN OINARRIZKO KONTZEPTUAK

Aldez aurretik zehaztutako argumentu lehenengo argumentua bada

Demagun ondorengo datuak ditugula:

```
val velocidadCaminando = velocidad(42.195, 8.27)
val velocidadEnBicicleta = velocidad(42.195, 2.85)
val velocidadManejando = velocidad(42.195, 0.37)
val velocidadVolando = velocidad(0.12, 0.01)
```

FUNTZIOEN OINARRIZKO KONTZEPTUAK

Distantzia 42.195 denez denentzat, hegazkinarentzat izan ezik, eguneratu dezagun gure kodea eta distantziari 42.195 balioa erantsiko diogu aldez aurretik zehaztutako argumentu bezala.

```
fun velocidad(distancia: Double = 42.195, tiempo: Double) = distancia / tiempo
```

Deiak honela geratu dira:

```
val velocidadCaminando = velocidad(8.27)
val velocidadEnBicicleta = velocidad(2.85)
val velocidadManejando = velocidad(0.37)
val velocidadVolando = velocidad(0.12, 0.01)
```

FUNTZIOEN OINARRIZKO KONTZEPTUAK

Honek **errorea** ematen du, zergatik?

8.27 esleitu nahi genion denboraren parametroari, honela:

```
fun velocidad(distancia: Double = 42.195, tiempo: Double) = distancia / tiempo  
  
val velocidadCaminando = velocidad(8.27)
```



Baina benetan egiten ari garena:

```
fun velocidad(distancia: Double = 42.195, tiempo: Double) = distancia / tiempo  
  
val velocidadCaminando = velocidad(8.27)
```



FUNTZIOEN OINARRIZKO KONTZEPTUAK

Kotlini denbora bidali nahi diogula esateko, honela adierazitako argumentuak erabiliko ditugu:

```
val velocidadCaminando = velocidad(tiempo = 8.27)
val velocidadEnBicicleta = velocidad(tiempo = 2.85)
val velocidadManejando = velocidad(tiempo = 0.37)
val velocidadVolando = velocidad(0.12, 0.01)
```

FUNTZIOEN OINARRIZKO KONTZEPTUAK

Adierazpen-gorputzak eta bloke-gorputzak

Adierazpen-gorputzak:

```
val pi = 3.14

fun circunferencia(radio: Double) = 2 * pi * radio
```

Bloke-gorputzak:

```
val pi = 3.14

fun circunferencia(radio: Double): Double {
    return 2 * pi * radio
}
```

FUNTZIOEN OINARRIZKO KONTZEPTUAK

```
fun circumferencia(radio: Double): Double {  
    return 2 * pi * radio  
}
```

lo que sea que se devuelva aquí debe coincidir con el tipo indicado aquí

Praktika onak

Adierazpen bakar bat ebaluatzen duen funtzio bat duzunean, Kotlinen ohikoa da **gorputzaren adierazpenak** erabiltzea bloke-gorputzaren ordeez.

Bloke-gorputzak dituzten *funtzioek* *lerro asko* izan ditzaketen arren, funtzioak hobeto ulertzen dira labur mantentzen direnean.

FUNTZIOEN OINARRIZKO KONTZEPTUAK

Emaitzarik gabeko funtzioak

Batzuetan, emaitza bat itzultzen ez duen funtzio bat nahiko genuke. Adibidez, denboran zeharreko gehikuntza kopuru bat duen aldagai bat dugu, kontagailu izenekoa. Gehikuntza() izeneko funtzioa sortuko dugu. Funtzio horri deitzen dugun bakoitzean, kontagailua baten handituko da, eta sententzia ondorengoa izanik: `kontagailua = kontagailua + 1`.

Adierazpen-gorputzak dituzten funtzioek adierazpen bakarra izan dezakete. Ezin dugu esaera bat erabili adierazpen-gorputz batean. Adibidez, ezin dugu hau egin errorea ematen baitu:

```
var contador = 0

fun incremento() = contador = contador + 1
```

FUNTZIOEN OINARRIZKO KONTZEPTUAK

Adierazpen-gorputza erabili beharrean, hobe da bloke-gorputza erabiltzea funtzio horretarako.

```
var contador = 0

fun incremento() {
    contador = contador + 1
}
```

Ziur aski konturatu zineten ez dugula itzulera-mota zehazten.

No hay tipo de retorno aquí

```
fun incremento() {
    contador = contador + 1
}
```


FUNTZIOEN OINARRIZKO KONTZEPTUAK

Harrituta egongo zarie funtzioan itzultzen buen balioa mota zehazten ez dugulako baina Kotlinek badu Unit izeneko mota berezi bat non **Bloke-gorputza** duen funtzio batean itzulera mota, automatikoki itzultzen da.

Unit Ez da zenbaki edo kate bat, generiko bat da eta batzutan erabilgarria izango da.